

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 12: Edge ML Deployment: Running DT, RF and SVM classifier on Raspberry Pi with Real Time Sensor Interface

Name of Student: ...[Bushra Nazir](#).....

Registration No.: ...[1071-F22](#).....

Date of Experiment: ...[7-5-2026](#).....

Submitted To: ...[Engr.Asad](#).....

Objectives

- Deploy a pre-trained Decision Tree (DT) classifier on a Raspberry Pi for real-time inference using an HC-SR04 ultrasonic distance sensor.
- Extend deployment to Random Forest (RF) and SVM classifiers and compare their real-time inference performance on embedded hardware.
- Interface physical output (LED indicator) with the Pi GPIO to provide an immediate visual response to model predictions.
- Understand the full Machine Learning pipeline: data collection → model training (PC) → serialization → transfer → deployment (Pi).
- Evaluate and compare the performance, latency, and resource usage of DT, RF, and SVM on a resource-constrained edge device.

Equipment Required

Component	Quantity / Details
Raspberry Pi4 Model B (or Pi 3B+)	1 unit
HC-SR04 Ultrasonic Distance Sensor	1 unit
LED (Red or Green 5mm)	1 unit
Push Button (Momentary SPST)	1 unit
220 Ω Resistor (LED current limiting)	1 unit
1 k Ω + 2 k Ω Resistors (voltage divider for Echo)	1 set
Jumper wires & Breadboard	Assorted
PC / Laptop with Python 3.8+, scikit-learn, joblib	1 unit
SD Card with Raspberry Pi OS Lite (64-bit)	16 GB+

What is Edge ML?

In Lab 11 we explored Decision Trees, Random Forests, and SVMs as software classifiers running on a standard PC. In this lab, we close the loop between software and hardware by

deploying those same trained models onto a Raspberry Pi a credit-card-sized Linux computer with physical GPIO pins.

This is known as Edge Machine Learning (Edge ML) or TinyML: running inference directly on the device that collects the data, rather than sending it to a cloud server. Edge ML offers:

- ✓ Low latency predictions in milliseconds without network roundtrips.
- ✓ Privacy raw sensor data never leaves the device.
- ✓ Offline operation no internet connection required.
- ✓ Low-cost commodity hardware (Pi, Arduino, microcontrollers) replaces expensive servers.

In this experiment, an HC-SR04 ultrasonic sensor continuously measures the distance to an object. The Raspberry Pi feeds this reading into our pre-trained classifier, which outputs a label ('Safe' or 'Unsafe'). A physical LED then illuminates to reflect the result giving us a real embedded ML system in minutes.

Hardware Setup

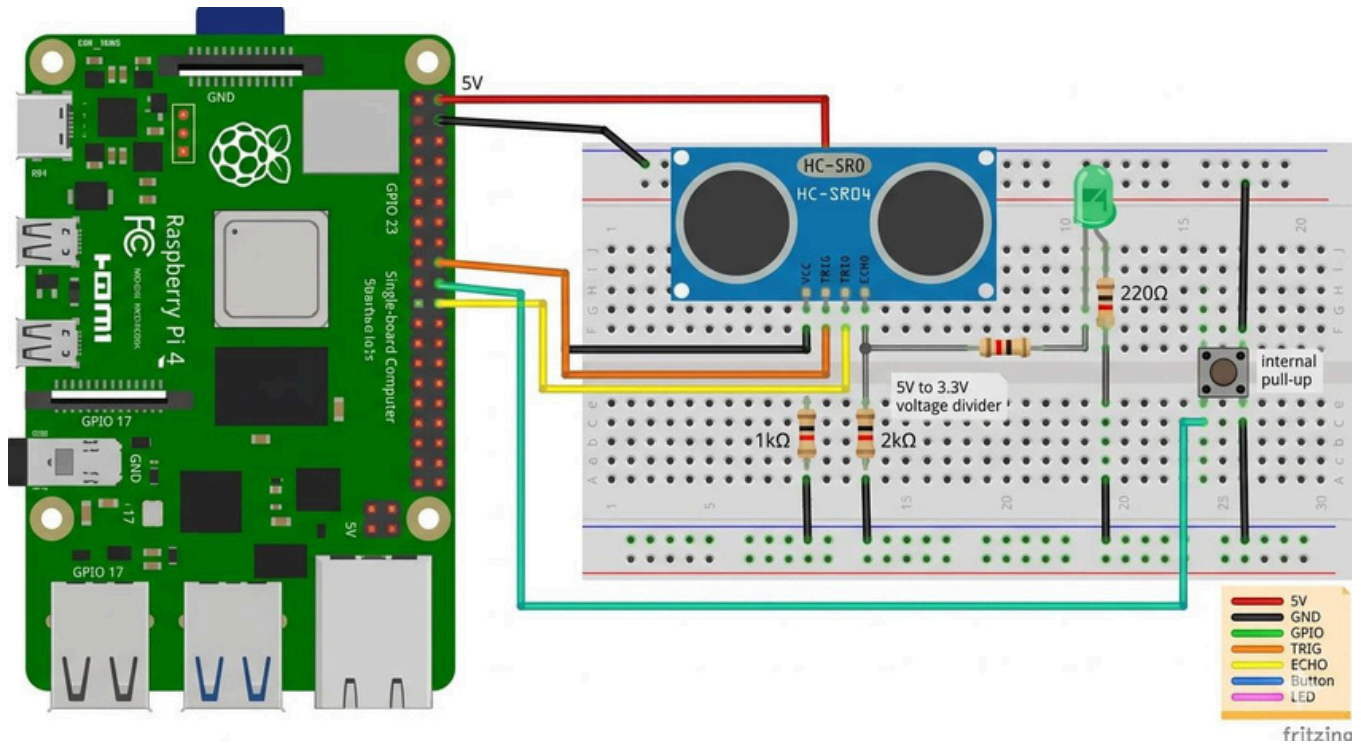
Component Overview

The HC-SR04 sensor emits an ultrasonic burst and measures the time for the echo to return. It communicates via two digital lines: TRIG (trigger pulse in) and ECHO (echo pulse out). The Pi calculates distance from the pulse width.

Voltage Warning

The HC-SR04 ECHOpin outputs 5 V logic. Raspberry Pi GPIO inputs are only 3.3 V tolerant. Always use a voltage divider (1 k Ω + 2 k Ω) on the ECHO line or you risk permanently damaging the Pi's GPIO.

Circuit Diagram



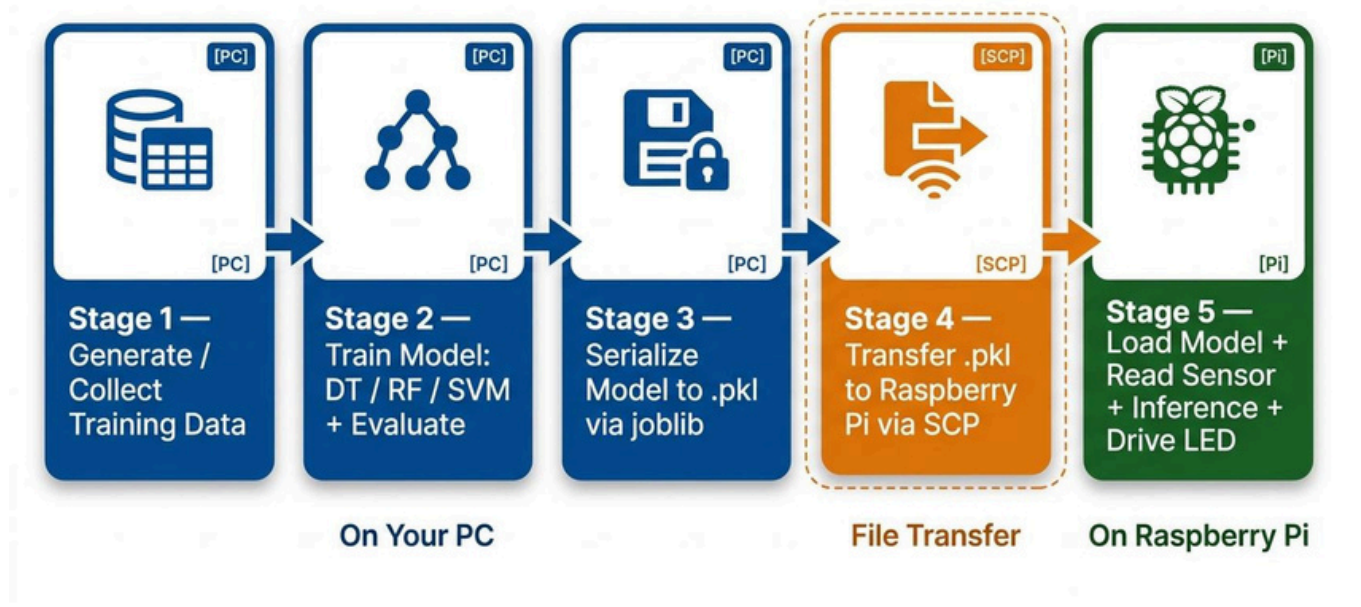
GPIO Connection Reference

* The ECHO pin requires a voltage divider: connect a 1 kΩ resistor between ECHO and GPIO 24, then a 2 kΩ resistor between GPIO 24 and GND.

Component / Pin	Pi Physical Pin	BCM	GPIO
HC-SR04 VCC	Pin 2 (5V)	5V	Power
HC-SR04 GND	Pin 6 (GND)	Ground	
HC-SR04 TRIG	Pin 16	GPIO	23
HC-SR04 ECHO	Pin 18 *	GPIO	24
LED Anode (+)	Pin 12	GPIO	18
LED Cathode (-)	Pin 20 (GND)	Ground	
Button Leg A	Pin 11	GPIO	17
Button Leg B	Pin 25 (GND)	Ground	

Software Pipeline

The full ML pipeline for embedded deployment has five stages. Complete each stage in order.



Step-by-Step Implementation

STEP 1 Train & Save the Model (on your PC)

Run the code below on your PC (Jupyter Notebook or Google Colab). It trains a Decision Tree on synthetic distance data, evaluates it, and saves the model to a .pkl file.

1. Decision Tree Training & Export

```
# — train_dt.py (run on PC) —————
import numpy as np
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# — 1. Synthetic sensor dataset —————
# Distance (cm) : label 1 = Safe (clear path), 0 = Unsafe (obstacle)
distances = np.array([
    10, 15, 20, 22, 18, 12, 8,          # Unsafe: obstacle detected
    40, 55, 60, 70, 80, 90, 100, 120,    # Safe: clear path
    25, 30, 35,                          # Borderline (Unsafe)
    50, 65, 75, 95, 110                  # Borderline (Safe)
]).reshape(-1, 1)
```

```

labels = np.array([
    0,0,0, 0,0,0, 0, 1,1,1,      #Unsafe
    1,1,1, 1,1, 0,0,0,      #Safe
    1,1,1,      # Unsafe(borderline)
                1,1      # Safe (borderline)
])

#—2.Train /test split—————
X_train, X_test, y_train, y_test = train_test_split(
    distances, labels, test_size=0.25, random_state=42
)

#—3.Train DecisionTree—————
dt=DecisionTreeClassifier(criterion='entropy',max_depth=4, random_state=42)
dt.fit(X_train, y_train)

#—4.Evaluate—————
y_pred = dt.predict(X_test)
print('=== Decision Tree Classification Report ===')
print(classification_report(y_test,y_pred,target_names=[ 'Unsafe', 'Safe' ]))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=[ 'Unsafe', 'Safe' ], yticklabels=[ 'Unsafe', 'Safe' ])
plt.title('Decision Tree Confusion Matrix'); plt.show()

#—5.Visualisethe tree—————
plt.figure(figsize=(10, 5))
plot_tree(dt, feature_names=[ 'Distance(cm)' ], class_names=[ 'Unsafe', 'Safe' ],
filled=True)
plt.title('TrainedDecisionTree');plt.tight_layout(); plt.show()

#—6.Save model—————
joblib.dump(dt, 'dt_model.pkl')
print('Model saved: dt_model.pkl')

```

2. Random Forest Training & Export

```

# — train_rf.py (run on PC) ———
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=50, random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
print('=== Random Forest Classification Report ===')

```

```
print(classification_report(y_test, y_pred_rf, target_names=['Unsafe', 'Safe']))

joblib.dump(rf, 'rf_model.pkl')
print('Model saved: rf_model.pkl')
```

3. SVM Training & Export

```
# — train_svm.py (run on PC) —————
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
import pickle

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

svm = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True)
svm.fit(X_train_s, y_train)

y_pred_svm = svm.predict(X_test_s)
print('=== SVM Classification Report ===')
print(classification_report(y_test, y_pred_svm, target_names=['Unsafe', 'Safe']))

# Save BOTH scaler and SVM both needed at inference time!
joblib.dump({'scaler': scaler, 'svm': svm}, 'svm_bundle.pkl')
print('Model saved: svm_bundle.pkl')
```

STEP 2 Transfer Model Files to Raspberry Pi

There are three ways to copy the .pkl file(s) from your PC to the Pi. Choose any one method:

1. SCP over SSH (recommended, same Wi-Fi)

#Run this in your PC terminal (not on the Pi)

#Replace <PI_IP> with the Pi's IP address (find it with: hostname -I)

```
scp dt_model.pkl rf_model.pkl svm_bundle.pkl pi@<PI_IP>:/home/pi/ml_lab/
```

2. USB Flash Drive

- Copy .pkl file to a USB drive on your PC.
- Insert USB into Pi, then: `sudo cp /media/pi/USB_NAME/*.pkl /home/pi/ml_lab/`

3. Google Drive + gdown

On Pi terminal:

```
pip install gdown
```

```
gdown 'https://drive.google.com/uc?id=<YOUR_FILE_ID>' -O  
/home/pi/ml_lab/dt_model.pkl
```

STEP 3 Prepare the Raspberry Pi Environment

Install Python Dependencies

Run these commands in a Pi terminal (SSH or directly):

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install python3-pip python3-venv -y
```

Create a virtual environment (best practice)

```
python3 -m venv ~/ml_env
```

```
source ~/ml_env/bin/activate
```

Install required packages

```
pip install RPi.GPIO joblib numpy scikit-learn
```

Create working directory

```
mkdir -p ~/ml_lab
```

```
cd ~/ml_lab
```

STEP 4 Build the Circuit

Follow the GPIO Connection Reference table on the previous page. Key points:

- NEVER skip the voltage divider on the ECHO pin 5V WILL damage the Pi.
- The LED current-limiting resistor (220 Ω) connects between GPIO 18 and the LED anode.
- The push button uses the Pi's internal pull-up; pressing it drives GPIO 17 LOW.
- Keep wires short and neat on the breadboard to avoid noise on the ECHO signal.

STEP 5 Deployment Code on Raspberry Pi

1. Decision Tree Deployment

Save this as dt_deploy.py in ~/ml_lab/ and run with: `python3 dt_deploy.py`

```
# — dt_deploy.py (run on Raspberry Pi) —————
```



```

import RPi.GPIO as GPIO
import time
import joblib
import numpy as np

# — GPIO configuration —————
TRIG    = 23    # HC-SR04 Trigger
ECHO    = 24    # HC-SR04 Echo   (via voltage divider!)
LED     = 18    # Green LED
BUTTON  = 17    # Push button

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(TRIG,    GPIO.OUT)
GPIO.setup(ECHO,    GPIO.IN)
GPIO.setup(LED,     GPIO.OUT)
GPIO.setup(BUTTON,  GPIO.IN, pull_up_down=GPIO.PUD_UP)

# — Load pre-trained Decision Tree —————
model = joblib.load('dt_model.pkl')
print('[INFO] Decision Tree model loaded successfully.')

# — Distance measurement function —————
def measure_distance():
    """Return distance in cm using HC-SR04 ultrasonic sensor."""
    GPIO.output(TRIG, False)
    time.sleep(0.05)                # Let sensor settle

    GPIO.output(TRIG, True)         # Send 10 µs trigger pulse
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    timeout = time.time() + 0.04    # 40 ms timeout
    pulse_start = time.time()
    while GPIO.input(ECHO) == 0:    # Wait for echo HIGH
        pulse_start = time.time()
        if time.time() > timeout:
            return None            # Sensor timeout

    pulse_end = time.time()
    while GPIO.input(ECHO) == 1:    # Wait for echo LOW
        pulse_end = time.time()
        if time.time() > timeout:
            return None

    duration = pulse_end - pulse_start
    distance = duration * 17150      # Speed of sound / 2
    return round(distance, 2)

```

```

#—Maininference loop—
print('[INFO] System ready.Press  buttontoclassifydistance.')
print('[INFO] Press Ctrl+C to exit.\n')

try:
    while True:
        ifGPIO.input(BUTTON)==GPIO.LOW:          #Buttonpressed
            dist = measure_distance()

            if dist is None:
                print('[WARN]Sensortimeout      checkwiring.')
                GPIO.output(LED, GPIO.LOW)
            elifdist >400ordist<2:                #Outofsensor range
                print(f'[WARN]Distance{dist}cmoutofsensor range.')
                GPIO.output(LED, GPIO.LOW)
            else:
                pred = model.predict([[dist]])[0]
                label = 'SAFE' if pred == 1 else 'UNSAFE'
                print(f'Distance: {dist:6.1f}cm → Prediction: {label}')
                GPIO.output(LED,GPIO.HIGH ifpred==1      else GPIO.LOW)

                time.sleep(0.5)    #Debouncedelay

                time.sleep(0.05)   #Pollinginterval

except KeyboardInterrupt:
    print('\n[INFO] Shutting down...')
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()
    print('[INFO] GPIO cleaned up. Goodbye!')

```

4. All Three Classifiers with Performance Timing

This advanced version loads all three models and lets you compare inference latency on the Pi in real time.

```

# — compare_deploy.py (run on Raspberry Pi) —
import RPi.GPIO as GPIO, time, joblib, numpy as np

TRIG = 23; ECHO = 24; LED = 18; BUTTON = 17
GPIO.setmode(GPIO.BCM); GPIO.setwarnings(False)
GPIO.setup(TRIG, GPIO.OUT); GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(LED, GPIO.OUT)
GPIO.setup(BUTTON, GPIO.IN, pull_up_down=GPIO.PUD_UP)

# — Load all three models —

```

```

dt = joblib.load('dt_model.pkl')
rf = joblib.load('rf_model.pkl')
svm_bundle = joblib.load('svm_bundle.pkl')
scaler = svm_bundle['scaler']
svm = svm_bundle['svm']
print('[INFO] All models loaded.')

def measure_distance():
    GPIO.output(TRIG, False); time.sleep(0.05)
    GPIO.output(TRIG, True); time.sleep(0.00001); GPIO.output(TRIG, False)
    t = time.time()
    while GPIO.input(ECHO) == 0:
        ps = time.time()
        if time.time() - t > 0.04: return None
    pe = time.time()
    while GPIO.input(ECHO) == 1:
        pe = time.time()
        if time.time() - t > 0.04: return None
    return round((pe - ps) * 17150, 2)

def infer_and_time(clf, X_input):
    """Run prediction and return (label, latency_ms)."""
    t0 = time.perf_counter()
    pred = clf.predict(X_input)[0]
    ms = (time.perf_counter() - t0) * 1000
    return ('SAFE' if pred == 1 else 'UNSAFE'), round(ms, 3)

print('\n[INFO] Press button to run all three classifiers.')
print('      Ctrl+C to quit.\n')
print(f'{{'Model':<16}} {{'Result':<10}} {{'Latency (ms)':>14}}')
print('-' * 44)

try:
    while True:
        if GPIO.input(BUTTON) == GPIO.LOW:
            dist = measure_distance()
            if dist is None: print('Sensor timeout.');
```

```

        ('Random Forest', rf_label, rf_ms),
        ('SVM(RBF)', svm_label, svm_ms)]:
    print(f'{name:<16} {lbl:<10}{ms:>14.3f} ms')

    # LED driven by majority vote
    votes=[dt_label,rf_label,svm_label].count('SAFE')
    GPIO.output(LED,GPIO.HIGHif votes >= 2 else GPIO.LOW)
    print(f'\nMajorityvote→{'SAFE' if votes >= 2 else 'UNSAFE ✗'}')
    time.sleep(0.7)

    time.sleep(0.05)

except KeyboardInterrupt:
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()
    print('\n[INFO] Done.')

```

Expected Terminal Output

When running compare_deploy.py and pressing the button with an object ~15 cm away, you should see:

```

[INFO] All models loaded.
[INFO] Press button to run all three classifiers.
      Ctrl+C to quit.

```

Model	Result	Latency (ms)

Sensor reading: 14.73 cm		

Decision Tree	UNSAFE	0.412 ms
Random Forest	UNSAFE	3.817 ms
SVM (RBF)	UNSAFE	1.203 ms

Majority vote → UNSAFE ✗

Sensor reading: 72.50 cm

Decision Tree	SAFE	0.388 ms
Random Forest	SAFE	3.645 ms
SVM (RBF)	SAFE	1.191 ms

Majority vote → SAFE ✓

Performance Comparison: DT vs RF vs SVM on Pi

The table below summarises the expected behaviour of each classifier when deployed on Raspberry Pi hardware (based on the Heart Disease dataset results from Lab 11):

Classifier	Accuracy	Overfitting Risk	Speed on Pi	Pi Deployable?
Decision Tree	~70%	High	Very Fast	✓Yes
Random Forest	~86%	Low	Moderate	✓Yes
SVM (RBF)	~89%	Low	Fast	✓Yes

Key Insight

Decision Trees have the fastest inference time on the Pi but suffer from overfitting. Random Forests are the best balanced choice. SVMs with RBF kernel achieve the highest accuracy and have surprisingly low latency on the Pi because sklearn's libsvm implementation is compiled C code.

Troubleshooting Guide

Problem	Solution
Sensor always returns None / timeout	Check TRIG is GPIO OUT, ECHO is GPIO IN. Verify 5V supply is present. Make sure voltage divider is correct.

RuntimeError: No access to /dev/mem	Run with sudo: <code>sudo python3 dt_deploy.py</code> , or add your user to the gpio group: <code>sudo usermod -aG gpio pi</code>
ValueError when loading .pkl	sklearn version mismatch. Match versions on PC and Pi, or retrain model on the Pi itself. Check LED polarity (flat side = cathode = GND).
LED stays OFF for all readings	Verify 220 Ω resistor in series. Test GPIO 18 manually: <code>GPIO.output(18, True)</code> . Confirm PUD_UP pull-up is set. Check button
Button press not detected	connects GPIO 17 to GND (not 3.3V) when pressed. Your training data may be too small. Try collecting
Model accuracy seems wrong	real sensor readings and retraining with at least 50 samples.

Lab Tasks

1. Single Model Deployment (DecisionTree)

1. Build the circuit as described in Step 4.
2. Train and export dt_model.pkl using the code in Step 1a on your PC.
3. Transfer dt_model.pkl to the Pi using any method from Step 2.
4. Run dt_deploy.py. Place an object closer than 30 cm LED should turn OFF (Unsafe).
5. Move the object beyond 40 cm LED should turn ON (Safe).
6. Record at least 10 distance readings and their predictions in the observation table below.

S.No.	Distance (cm)	DT Prediction	RF Prediction	SVM Prediction
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

2. Three-Model Comparison Dashboard

1. Train and export rf_model.pkl and svm_bundle.pkl using the code in Steps 1b and 1c.
2. Transfer all three .pkl files to the Pi.
3. Run compare_deploy.py and observe the inference latency for each classifier.
4. Record which classifier gives the lowest and highest inference latency.
5. Do all three classifiers agree on every reading? If not, which one differed?

3. Self-Collected Dataset & Retrain

1. Modify dt_deploy.py to log (distance, manual_label) pairs to a CSV file instead of predicting. Example: label 1 if you say 'safe', 0 if you say 'unsafe'.

2. Collect at least 30 real sensor readings at various distances.
3. Transfer the CSV to your PC and retrain a Decision Tree on your real data.
4. Compare the accuracy of the retrained model vs. the original synthetic-data model.
5. Discuss: Why is real-world training data usually better than synthetic data?

Code:

```
import numpy as np
import joblib

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

import matplotlib.pyplot as plt
import seaborn as sns

# Helper function to plot confusion matrix
def plot_confusion_matrix(y_true, y_pred, title):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(4, 3))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
                xticklabels=['Unsafe', 'Safe'], yticklabels=['Unsafe', 'Safe'])
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.title(title)
    plt.show()

# === Synthetic Dataset ===
distances = np.array([
    10, 15, 20, 22, 18, 12, 8,    # Unsafe
    40, 55, 60, 70, 80, 90, 100, 120, # Safe
    25, 30, 35,    # Borderline Unsafe
    50, 65, 75, 95, 110 # Borderline Safe
]).reshape(-1, 1)
```

```

labels = np.array([
    0,0,0,0,0,0,0,    # Unsafe
    1,1,1,1,1,1,1,1, # Safe

    0,0,0,            # Borderline Unsafe
    1,1,1,1,1        # Borderline Safe
])

# Train/Test Split

X_train, X_test, y_train, y_test = train_test_split(distances, labels,
test_size=0.25, random_state=42)
print("Data preparation complete. Training and testing sets created.")

print(f"Training data shape: {X_train.shape}, {y_train.shape}")
print(f"Testing data shape: {X_test.shape}, {y_test.shape}")

... Data preparation complete. Training and testing sets created.
Training data shape: (17, 1), (17,)
Testing data shape: (6, 1), (6,)

```

2. Decision Tree Classifier

Code:

```

# === Decision Tree ===

dt = DecisionTreeClassifier(criterion='entropy', max_depth=4, random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)

print("Decision Tree Report:\n", classification_report(y_test, y_pred_dt))
plot_confusion_matrix(y_test, y_pred_dt, 'Decision Tree Confusion Matrix')
plt.figure(figsize=(10, 8))

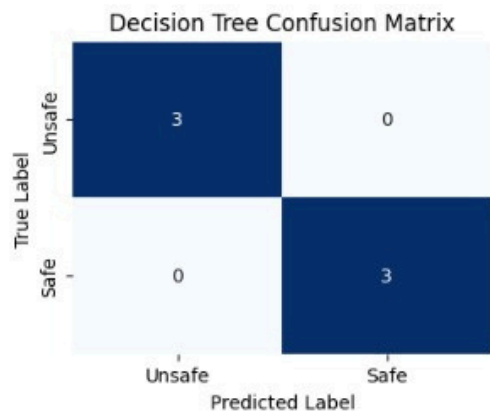
plot_tree(dt, filled=True, feature_names=['Distance'], class_names=['Unsafe',
'Safe'], rounded=True)
plt.title('Decision Tree Visualization')

plt.show()

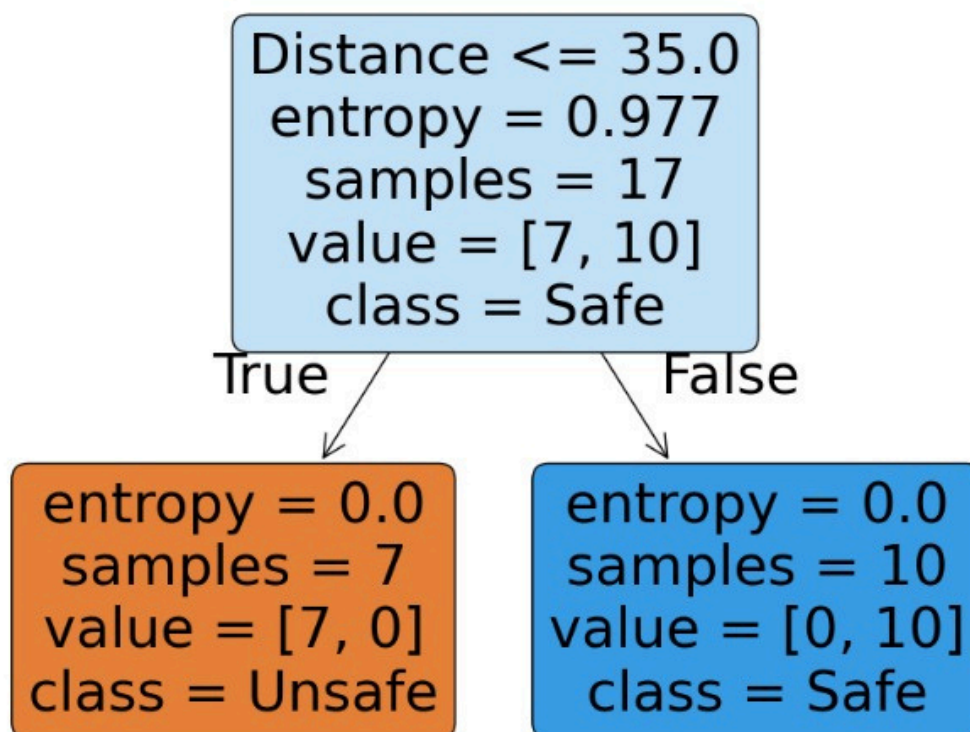
```

```
joblib.dump(dt, "dt_model.pkl")
```

```
print("Decision Tree model trained, evaluated, visualized, and saved as  
dt_model.pkl")
```



Decision Tree Visualization



3. Random Forest Classifier

```
# === Random Forest ===
```

```
rf = RandomForestClassifier(n_estimators=50, random_state=42)
```

```
rf.fit(X_train, y_train)
```

```
y_pred_rf = rf.predict(X_test)
```

```

print("Random Forest Report:\n", classification_report(y_test, y_pred_rf))
plot_confusion_matrix(y_test, y_pred_rf, 'Random Forest Confusion Matrix')
joblib.dump(rf, "rf_model.pkl")
print("Random Forest model trained, evaluated, and saved as rf_model.pkl")

```

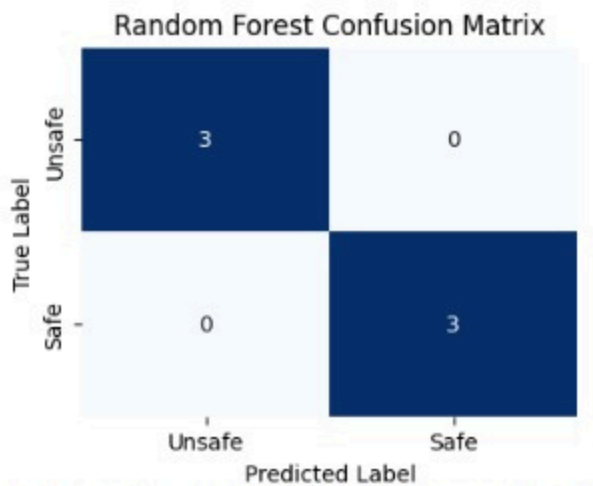
```

... Random Forest Report:
              precision    recall  f1-score   support

     0       1.00      1.00      1.00         3
     1       1.00      1.00      1.00         3

 accuracy          1.00
 macro avg          1.00
 weighted avg       1.00

```



Random Forest model trained, evaluated, and saved as rf_model.pkl

4. Support Vector Machine (SVM) Classifier

```

# === SVM (with scaling) ===

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

svm = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True)
svm.fit(X_train_s, y_train)
y_pred_svm = svm.predict(X_test_s)

print("SVM Report:\n", classification_report(y_test, y_pred_svm))
plot_confusion_matrix(y_test, y_pred_svm, 'SVM Confusion Matrix')

```

```
# Save both scaler + model
```

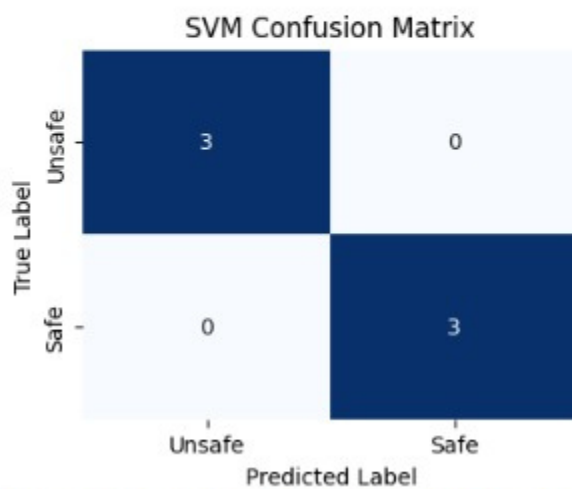
```
joblib.dump({"scaler": scaler, "svm": svm}, "svm_bundle.pkl")
```

```
print("SVM model (with its scaler) trained, evaluated, and saved as  
svm_bundle.pkl")
```

```
SVM Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	3
1	1.00	1.00	1.00	3
accuracy			1.00	6
macro avg	1.00	1.00	1.00	6
weighted avg	1.00	1.00	1.00	6



SVM model (with its scaler) trained, evaluated, and saved as svm_bundle.pkl

5. Conclusion

```
print("✅ All models (Decision Tree, Random Forest, SVM) have been trained, evaluated, and saved.")
```

```
✅ All models (Decision Tree, Random Forest, SVM) have been trained, evaluated, and saved.
```

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 12

Edge ML Deployment: Running DT, RF and SVM classifier on Raspberry Pi with Real Time Sensor Interface

Name: Bushra Nazir Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) <u>PSYCHOMOTOR</u> (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) <u>COGNITIVE</u> (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) <u>AFFECTIVE</u> (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) : 10

Marks Obtained (PO + CO + AO) :

Instructor